SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

R PROGRAMMING

Matrices • Arrays • S3, S4 and Reference Classes

Student Name	Paleru Dharani Govardhan
Programme / Branch	B.Tech – Artificial Intelligence and Machine Learning (AIML)
Institution	SIMATS Engineering College, Saveetha University
Course Name	R Programming
Course Code	ITA04

.

Topic 1: Creating Matrices and Operations on Matrices

Q1. Quarterly product sales

Use byrow = TRUE so the values fill across each product row. MARGIN = 1 applies the function to rows; MARGIN = 2 applies it to columns.

```
sales <- c(120,150,130,160,90,110,100,115,200,210,190,220)
S <- matrix(sales, nrow = 3, byrow = TRUE)
rownames(S) <- paste0("Product", 1:3)
colnames(S) <- paste0("Q", 1:4)
print(S)
product_totals <- apply(S, 1, sum)
quarter_averages <- apply(S, 2, mean)
product_totals
quarter_averages
```

Expected result / note: Product totals: Product1 = 560, Product2 = 415, Product3 = 820. Quarter averages: Q1 = 136.67, Q2 = 156.67, Q3 = 140, Q4 = 165.

Q2. Element-wise multiplication versus matrix multiplication

A is 2 × 3 and B is 3 × 2. Element-wise multiplication requires conformable dimensions (or recycling, which should not be relied on); matrix multiplication requires the number of columns of A to equal the number of rows of B.

```
A <- matrix(1:6, nrow = 2)
B <- matrix(1:6, nrow = 3)
A %*% B                # 2 x 2 result
B_for_elementwise <- matrix(1:6, nrow = 2)
A * B_for_elementwise   # 2 x 3 element-wise result
```

Expected result / note: The matrix product is matrix(c(22,28,49,64), nrow = 2) in R's column-major display; its dimensions are 2 × 2.

Q3. Transpose, determinant and inverse

Use t(), det() and solve(). The determinant is -1, so the matrix is invertible. round() removes tiny floating-point display differences.

```
M <- matrix(c(2,1,1, 1,3,2, 1,0,0), nrow = 3, byrow = TRUE)
t(M)
det(M)
M_inv <- solve(M)
M_inv
round(M %*% M_inv, 6)
```

Expected result / note: The final product is the 3 × 3 identity matrix, up to numerical precision.

Q4. Solve a system of linear equations

The coefficient matrix is A = [[2,1,1],[1,3,2],[1,0,0]]. The solution is x = 1, y = 2, z = 3. Replacing row 3 with twice row 2 makes two rows dependent, so the determinant becomes zero and the system has no unique solution.

```
A <- matrix(c(2,1,1, 1,3,2, 1,0,0), nrow = 3, byrow = TRUE)
b <- c(7,13,1)
solve(A, b)
A_bad <- A
A_bad[3, ] <- c(2,6,4)
det(A_bad)             # 0
solve(A_bad, b)        # error: computationally singular
```

Expected result / note: A singular matrix has rank less than 3; its inverse does not exist.

Q5. Indexing, filtering and extending a matrix

```
X <- matrix(1:12, nrow = 3)
X[2, ]              # second row
X[, 3]              # third column
X[1:2, 2:4]          # selected sub-matrix
X[X > 6]             # values greater than 6
X2 <- rbind(X, c(13,14,15,16))
dim(X2)             # 4 4
X3 <- cbind(X2, NewCol = c(17,18,19,20))
dim(X3)             # 4 5
```

R indexes matrices as [row, column]. After adding a row the dimensions are 4 × 4; after adding a column they are 4 × 5.

Q6. Recycling in rbind()

m has three columns, while 7:8 has two elements. R recycles the vector to fill the row, producing 7, 8, 7 and a warning because the row length is not a multiple of the vector length. The length-three vector 7:9 fits exactly.

```
m <- matrix(1:6, nrow = 2)
rbind(m, 7:8)        # warns; appended row is 7 8 7
rbind(m, 7:9)        # appended row is 7 8 9
```

Expected result / note: Check the number of values before binding rows or columns to prevent unintended recycling.

Q7. Diagonal, trace and upper triangle

```
M <- matrix(1:16, nrow = 4)
diag(M)
sum(diag(M))          # trace = 34
M[upper.tri(M)]       # entries above the diagonal
M2 <- M
M2[lower.tri(M2)] <- 0
M
M2
```

upper.tri(M) returns a TRUE/FALSE logical matrix identifying positions strictly above the main diagonal.

Q8. apply(), rowSums() and colSums()

With MARGIN = 1, apply works across rows; MARGIN = 2 works across columns. rowSums() and colSums() are specialised and generally faster for sums. A user-defined function is useful for statistics such as range (maximum minus minimum).

```
M <- matrix(1:9, nrow = 3)
apply(M, 1, sum)
apply(M, 2, sum)
rowSums(M)
colSums(M)
apply(M, 1, function(x) max(x) - min(x))
```

Expected result / note: Row sums: 12, 15, 18. Column sums: 6, 15, 24. Each returns a numeric vector.

Q9. Student totals, percentages and subject averages

```
# marks is a 5 x 3 numeric matrix
marks <- matrix(c(75,60,45,90,55,
                  80,70,50,85,65,
                  70,75,40,95,60), nrow = 5)
colnames(marks) <- c("Subject1", "Subject2", "Subject3")
marks <- cbind(marks, Total = rowSums(marks))
marks <- cbind(marks, Percentage = rowSums(marks[,1:3]) / 300 * 100)
```

```
eligible <- which(marks[, "Percentage"] >= 60)
subject_averages <- colMeans(marks[, 1:3])
marks
eligible
subject_averages
```

The output has five rows and five columns: three subject marks, Total, and Percentage. eligible contains row indices of students scoring at least 60%; subject_averages contains one mean per subject.

Q10. Type coercion in a matrix

Combining character labels with a numeric matrix using cbind() coerces the combined matrix to character because an atomic matrix can store only one basic type. Arithmetic such as rowSums() then fails or cannot operate meaningfully on the character data. Use a data frame for mixed types.

```
marks <- matrix(1:15, nrow = 5)
result <- c("Pass","Fail","Pass","Pass","Fail")
mixed <- cbind(marks, Result = result)
typeof(mixed)          # "character"
# rowSums(mixed) would fail because values are character
student_data <- data.frame(marks, Result = result)
```

Expected result / note: A data.frame stores each column with its own type, so numeric marks remain numeric and Result remains character.

Topic 2: Creating Arrays and Operations on Arrays

Q1. Create and index a three-dimensional array

```
dn <- list(c("R1","R2","R3"), c("C1","C2","C3","C4"), c("Sheet1","Sheet2"))
a <- array(1:24, dim = c(3,4,2), dimnames = dn)
a
dim(a)                 # 3 4 2
length(a)              # 24
a[2,3,2]
a["R2","C3","Sheet2"]
```

Both indexing expressions return the same element: 20.

Q2. Sales totals by branch, quarter and year

Branch totals retain dimension 1, quarter totals retain dimension 2, and yearly totals retain dimension 3. The other dimensions are summed away.

```
sales <- array(1:24, dim = c(3,4,2))
apply(sales, 1, sum)    # branch totals
apply(sales, 2, sum)    # quarter totals
apply(sales, 3, sum)    # year totals
```

Expected result / note: Expected totals: branches = 92, 100, 108; quarters = 48, 66, 84, 102; years = 78, 222.

Q3. Array indexing and dropped dimensions

By default, R drops dimensions of length one. a[1,,] is a 3 × 4 matrix, a[,2,3] is a vector of length 2, and a[2,3,4] is a single numeric value. drop = FALSE preserves the selected dimensions.

```
a <- array(1:24, dim = c(2,3,4))
dim(a[1,,])            # 3 4
dim(a[1,,, drop = FALSE]) # 1 3 4
a[,2,3]                # vector: 15 16
a[2,3,4]               # 24
```

Expected result / note: Preserving dimensions is useful when passing subsets into functions that expect an array or matrix.

Q4. Array arithmetic and logical indexing

```
a1 <- array(1:12, dim = c(2,3,2))
a2 <- array(12:1, dim = c(2,3,2))
a1 + a2
a1 * a2
comparison <- a1 > a2
comparison
sum(comparison)          # number of TRUE values
which(a1 == 6)           # linear index
```

The element-wise sum is an array of 13s. Exactly 6 positions have a1 > a2; a1 == 6 occurs at linear position 6.

Q5. Array recycling and excess values

dim = c(2,3,2) requires 12 elements. array(1:5, ...) recycles 1:5 to fill 12 positions; the second layer is filled with 2,3,4,5,1,2 in column-major order. array(1:30, ...) uses only the first 12 values and discards the rest.

```
a_small <- array(1:5, dim = c(2,3,2))
a_large <- array(1:30, dim = c(2,3,2))
a_small[,,2]
a_large[,,2]
```

Expected result / note: Recycling and truncation can conceal data-length mistakes; explicitly check length(data) against prod(dimensions) when exact matching is required.

Q6. Permuting array dimensions

```
a <- array(1:24, dim = c(2,3,4))
b <- aperm(a, c(2,1,3))
dim(b)                   # 3 2 4
a[1,2,3] == b[2,1,3]     # TRUE
c <- aperm(a, c(3,1,2))
dim(c)                   # 4 2 3
```

aperm() permutes any number of array dimensions, while t() transposes a two-dimensional matrix.

Q7. Combining matrices into an array

R stores matrix values column-wise. c(m1, m2) concatenates all of m1 first, followed by all of m2, so the two 2 × 3 layers in the array correspond to m1 and m2. cbind() joins matrices by columns; rbind() joins them by rows.

```
m1 <- matrix(1:6, nrow = 2)
m2 <- matrix(7:12, nrow = 2)
a <- array(c(m1, m2), dim = c(2,3,2))
a[,,1]                   # m1
a[,,2]                   # m2
cbind(m1, m2)
rbind(m1, m2)
is.matrix(m1)            # TRUE
is.array(m1)             # TRUE
```

Expected result / note: A matrix is a two-dimensional array, so is.array() is TRUE for matrices.

Q8. City temperatures

```
# temp has dimensions city x day x season
city_means <- apply(temp, 1, mean)
season_maxima <- apply(temp, 3, max)
day_of_max_season1 <- which.max(temp[,,1], arr.ind = TRUE)
```

```
# row of arr.ind identifies city; column identifies day
city2_season2 <- temp[2,,2]
city_means
season_maxima
day_of_max_season1
city2_season2
```

> Because actual temperature values were not supplied, the exact numeric results depend on the contents of temp. To get the day number only when flattening a 3 × 7 slice, use which.max(temp[,,1]) and interpret its column-major index carefully.

Q9. apply() with different MARGIN values

apply(arr, 3, sum) retains the third dimension and returns four layer sums. apply(arr, c(1,2), sum) retains the first two dimensions and sums over the third. apply(arr, c(1,3), sum) retains dimensions 1 and 3 and sums over dimension 2.

```
arr <- array(1:24, dim = c(2,3,4))
apply(arr, 3, sum)      # 21, 57, 93, 129
apply(arr, c(1,2), sum)  # 2 x 3 matrix
apply(arr, c(1,3), sum)  # 2 x 4 matrix
```

Expected result / note: The first result is c(21, 57, 93, 129); the second is a 2 × 3 matrix with rows (40, 48, 56) and (44, 52, 60); the third is a 2 × 4 matrix. MARGIN specifies the dimensions retained.

Q10. Modify profit/loss values

Count changed elements before editing. Negative values are replaced with zero first, then the entire second layer is multiplied by 10.

```
arr <- array(c(-3,5,-1,8,2,-7,4,-6,9,-2,1,-5), dim = c(2,3,2))
before_sum <- sum(arr)
negative_count <- sum(arr < 0)
arr[arr < 0] <- 0
arr[,,2] <- arr[,,2] * 10
after_sum <- sum(arr)
arr
before_sum
after_sum
negative_count
```

Expected result / note: Six negative values are changed to zero. The original sum is 5; after replacing negatives with zero and multiplying the second layer by 10, the total is 155.

Topic 3: Classes in R – S3, S4 and Reference Classes

Q1. S3 student class and methods

```
student <- function(name, rollno, marks) {
  structure(list(name = name, rollno = rollno, marks = marks),
            class = "student")
}
print.student <- function(x, ...) {
  cat("Name:", x$name, "\nRoll No:", x$rollno, "\nMarks:", x$marks, "\n")
  invisible(x)
}
summary.student <- function(object, ...) {
  if (object$marks >= 90) "A"
  else if (object$marks >= 75) "B"
  else if (object$marks >= 60) "C"
  else "Needs Improvement"
```

```
}
s1 <- student("Asha", 101, 92)
s2 <- student("Ravi", 102, 78)
print(s1); print(s2)
summary(s1); summary(s2)
```

print() and summary() are generic functions. R dispatches to print.student and summary.student based on the object's class attribute.

Q2. S3 method dispatch and NextMethod()

Objects of class student use info.student; teacher objects use info.teacher; a numeric vector uses info.default. For class c('grad_student','student'), R tries info.grad_student first; NextMethod() continues to info.student if that method exists.

```
info <- function(x) UseMethod("info")
info.student <- function(x) { cat("student method\n") }
info.teacher <- function(x) { cat("teacher method\n") }
info.default <- function(x) { cat("default method\n") }
info.grad_student <- function(x) {
  cat("graduate student method\n")
  NextMethod()
}
g <- structure(list(), class = c("grad_student", "student"))
info(g)
```

Expected result / note: Printed order for g: graduate student method, then student method. A numeric vector prints default method.

Q3. Employee and manager pay methods

```
pay <- function(x) UseMethod("pay")
pay.employee <- function(x) x$salary
pay.manager <- function(x) NextMethod() * 1.10
employee <- structure(list(name = "Mina", salary = 50000), class = "employee")
manager <- structure(list(name = "Arun", salary = 70000),
                     class = c("manager", "employee"))
pay(employee)          # 50000
pay(manager)           # 77000
```

NextMethod() obtains the employee salary, and the manager method adds a 10% bonus.

Q4. S3 validation compared with S4

S3 permits arbitrary list contents because it mainly uses a class attribute and does not enforce field types. A constructor plus validator can impose checks; S4 provides slot types and optional validity methods.

```
new_student <- function(name, marks) {
  x <- structure(list(name = name, marks = marks), class = "student")
  validate_student(x)
  x
}
validate_student <- function(x) {
  stopifnot(is.character(x$name), length(x$name) == 1,
            is.numeric(x$marks), length(x$marks) == 1)
  x
}
new_student("Ravi", 85)
# new_student("Ravi", "abc") fails validation
```

Expected result / note: S4 checks slot types when objects are created and can use a validity function for additional rules.

Q5. Define an S4 Person class

```
setClass("Person", slots = c(name = "character", age = "numeric"))
p <- new("Person", name = "Asha", age = 20)
p@age <- 21
show(p)
setGeneric("greet", function(object) standardGeneric("greet"))
setMethod("greet", "Person", function(object) {
  cat("Hello,", object@name, "\n")
})
greet(p)
is(p, "Person")
```

The @ operator accesses or changes slots. is(p, 'Person') returns TRUE.

Q6. S4 validity and negative age

If the validity function rejects negative ages, new('Person', name='Asha', age=-5) fails validity checking. Direct slot assignment may leave an object invalid until validObject(p) is called; validObject() reruns validity checks and reports the violation.

```
setClass("PersonChecked",
  slots = c(name = "character", age = "numeric"),
  validity = function(object) {
    if (object@age < 0) return("age must be non-negative")
    TRUE
  })
# new("PersonChecked", name = "Asha", age = -5) -> validity error
p <- new("PersonChecked", name = "Asha", age = 20)
p@age <- -10
validObject(p)          # reports invalid object
```

Expected result / note: S4 has stricter structural and validity support than S3, although explicit validity checks may be needed after slot modification.

Q7. S4 inheritance: Student extends Person

```
setClass("Person", slots = c(name = "character", age = "numeric"))
setClass("Student", contains = "Person", slots = c(marks = "numeric"))
setMethod("show", "Student", function(object) {
  callNextMethod()
  cat("Marks:", object@marks, "\n")
})
obj <- new("Student", name = "Asha", age = 20, marks = 88)
show(obj)
is(obj, "Person")        # TRUE
slotNames(obj)
```

Student inherits name and age from Person and adds marks; slotNames() lists all inherited and local slots.

Q8. Mutable Reference Class Account

```
Account <- setRefClass("Account",
  fields = list(balance = "numeric"),
  methods = list(
    deposit = function(amount) {
      balance <<- balance + amount
      invisible(balance)
    },
    withdraw = function(amount) {
```

```
    if (amount > balance) {
      cat("Insufficient balance\n")
    } else {
      balance <<- balance - amount
    }
    invisible(balance)
  }
 ))
acc <- Account$new(balance = 1000)
acc$deposit(500); acc$balance   # 1500
acc$withdraw(2000); acc$balance # 1500; withdrawal refused
acc$withdraw(300); acc$balance  # 1200
```

The balance remains 1500 after the rejected withdrawal and becomes 1200 after withdrawing 300.

Q9. Reference aliasing and copying

b <- a creates another reference to the same mutable object, so a change through a is visible through b. a$copy() creates a separate object c, which retains its original balance.

```
a <- Account$new(balance = 1000)
b <- a
c <- a$copy()
a$deposit(500)
a$balance             # 1500
b$balance             # 1500
c$balance             # 1000
```

Expected result / note: Reference classes have reference semantics; ordinary S3/S4 objects generally behave more like values and use copy-on-modify semantics.

Q10. Choosing between S3, S4 and Reference Classes

S3 is lightweight and flexible but does not enforce slot types. S4 defines formal slots, inheritance and validity checks, but objects are generally value-like. Reference Classes provide mutable objects and methods attached to objects, making them useful when account state changes repeatedly. For a bank application requiring state changes, account inheritance and strict types, Reference Classes with validation checks are a practical fit; S4 is also a strong choice for strict typed hierarchies if mutation is managed explicitly.

```
# Short outline:
Account <- setRefClass("Account",
  fields = list(balance = "numeric"),
  methods = list(
    deposit = function(amount) {
      stopifnot(is.numeric(amount), length(amount) == 1, amount >= 0)
      balance <<- balance + amount
    }
  ))
SavingsAccount <- setRefClass("SavingsAccount", contains = "Account",
  fields = list(rate = "numeric"))
```

Expected result / note: Drawback: Reference Classes allow shared mutable state, so aliases can change the same account unexpectedly unless references and operations are carefully controlled.